1. What is a CI/CD Pipeline?

Before understanding CI/CD, first understand this problem:

Imagine 50 developers are working on the same application.

- One develops login
- One develops payment
- One develops search
- One develops notifications

Now all their code must be:

- combined
- tested
- checked for errors
- deployed safely

Doing this manually every day is very difficult.

That's why companies use **CI/CD pipelines**.

Full Form

- **CI** → Continuous Integration
- **CD** → Continuous Delivery / Continuous Deployment

Core Idea of CI/CD

CI/CD is an **automated process** that helps software move from:

Developer Code → Testing → Deployment → Live Application

without too much manual work.

Understanding Continuous Integration (CI)

What happens in CI?

Developers continuously add their code to a shared repository.

Example:

- GitHub
- GitLab

- Bitbucket

Whenever code is pushed:

1. Code gets merged
2. Build process starts
3. Automated tests run
4. Errors are detected early

This is called **Continuous Integration**.

Why CI is Important

Without CI:

- one developer's code may break another's
- bugs may be found very late
- integration becomes difficult

CI helps:

- detect bugs early
- maintain code quality
- ensure all modules work together

What is Build?

Before moving ahead, understand "build".

A build means:

- converting source code into a runnable application

Example:

- Java code → .jar
- React app → optimized web files
- Python project → deployable package

Understanding Continuous Delivery (CD)

After CI succeeds:

The application is prepared for deployment.

This includes:

- packaging
- configuration
- deployment scripts
- environment setup

The software becomes “ready to release”.

This is Continuous Delivery.

Continuous Deployment

In some companies:

If all tests pass,
the application automatically becomes live.

No manual approval.

This is Continuous Deployment.

Real CI/CD Pipeline Flow

Developer writes code
↓
Pushes to GitHub
↓
CI Pipeline Starts
↓
Code Compilation
↓
Automated Testing
↓
Build Creation
↓
Deploy to DEV
↓
Deploy to QA
↓
Deploy to UAT
↓
Deploy to Production

Important Components in CI/CD

1. Source Control System

Stores code versions.

Examples:

- GitHub
- GitLab

Purpose:

- track changes
- collaborate safely

2. Build Server

Creates the application build.

Examples:

- Jenkins
- GitHub Actions

3. Automated Testing

Runs tests automatically.

Checks:

- functionality
- bugs
- performance

4. Deployment System

Deploys application to servers.

Could deploy to:

- cloud
- containers
- Kubernetes clusters

Popular CI/CD Tools

Tool	Purpose
Git	Version control
GitHub	Code repository
Jenkins	Automation
Docker	Containerization
Kubernetes	Container orchestration
Azure DevOps	CI/CD platform

Simple Real-Life Analogy

Think of a car manufacturing factory.

Manual Process

Workers manually:

- assemble parts
- inspect quality
- move vehicles

Slow and error-prone.

Automated Factory (CI/CD)

Machines automatically:

- assemble
- test
- move cars

Faster and more reliable.

CI/CD works similarly for software.

2. How Does an Application/Product Go Live?

This explains the complete software lifecycle.

Step 1 — Requirement Gathering

Business team talks with client.

They decide:

- what features are needed
- business goals
- user requirements

Example:
Food delivery app needs:

- login
- order tracking
- payment gateway

Step 2 — Planning & Architecture

Architects design:

- system structure
- database
- APIs
- scalability

Questions:

- Which language?
- Which database?
- Cloud or on-premise?

Step 3 — UI/UX Design

Designers create:

- screen layouts
- colors

- navigation
- user experience

Tools:

- Figma
- Adobe XD

Step 4 — Development Phase

Developers start coding.

Frontend:

- React
- Angular

Backend:

- Python
- Java
- Node.js

Database:

- MySQL
- PostgreSQL

Step 5 — Version Control

Code is stored in Git repositories.

Developers create:

- branches
- commits
- pull requests

Purpose:

- collaboration
- tracking changes
- rollback capability

Step 6 — CI/CD Pipeline

Once code is pushed:

Pipeline automatically:

- builds application
- runs tests
- checks quality
- creates deployable package

Step 7 — Deployment to Different Environments

Application moves through environments:

Local → DEV → QA → UAT → Production

Each environment has a purpose.

Step 8 — Testing

Testing teams validate:

- functionality
- security
- performance
- integration

Testing types:

- unit testing
- integration testing
- regression testing
- performance testing

Step 9 — User Acceptance Testing (UAT)

Business users or clients test application.

Purpose:

- confirm business requirements are satisfied

Step 10 — Production Deployment

Application is deployed to live servers.

Now real users access it.

This stage is called:

- Go Live
- Production Release

Step 11 — Monitoring & Maintenance

After release:

Teams monitor:

- crashes
- server load
- performance
- security threats

Monitoring tools:

- Grafana
- Prometheus
- Splunk

Important Concept — Rollback

If deployment fails:

System returns to previous stable version.

This is called rollback.

Very important in production systems.

3. Different Environments in a Project

Applications never directly go to production.

They move through controlled stages called environments.

Environment Flow

Local
↓
Development (DEV)
↓
Testing / QA
↓
UAT / Staging
↓
Production

1. Local Environment

Developer's own machine.

Purpose:

- coding
- debugging
- initial testing

Example:
Running Python app on laptop.

2. Development Environment (DEV)

Shared by development team.

Purpose:

- combine features
- integration testing

Characteristics:

- unstable sometimes
- frequent changes

3. QA / Testing Environment

Used by testers.

Purpose:

- detect bugs

- validate application behavior

Testing includes:

- functional testing
- regression testing
- API testing

4. UAT / Staging Environment

Almost identical to production.

Purpose:

- final business validation

Used by:

- clients
- product owners
- stakeholders

Difference Between QA and UAT

QA	UAT
Technical testing	Business validation
Done by testers	Done by clients/business
Finds bugs	Checks business expectations

5. Production Environment

Live environment.

Real users use the application here.

Requires:

- high security
- high stability
- monitoring
- backups

Why Multiple Environments Exist

Imagine directly deploying untested code to banking software.

Possible issues:

- money transfer failures
- crashes
- security risks

Multiple environments reduce risk.

Example of Complete Workflow

Suppose Instagram adds a new feature.

Workflow

Local

Developer builds feature.

↓

DEV

Feature merged with other developers' code.

↓

QA

Testing team validates functionality.

↓

UAT

Business team approves feature.

↓

Production

Feature becomes available to users worldwide.